Week 15 Final Assignment

Parsing Cisco Configs in Python - A Tutorial Script
Author: Ivan Lesley R. Mercado
Course: LIS875

Topic: Walking parent-child config blocks with `CiscoConfParse`

Tools used:
- Python 3
- CiscoConfParse
- VS Code

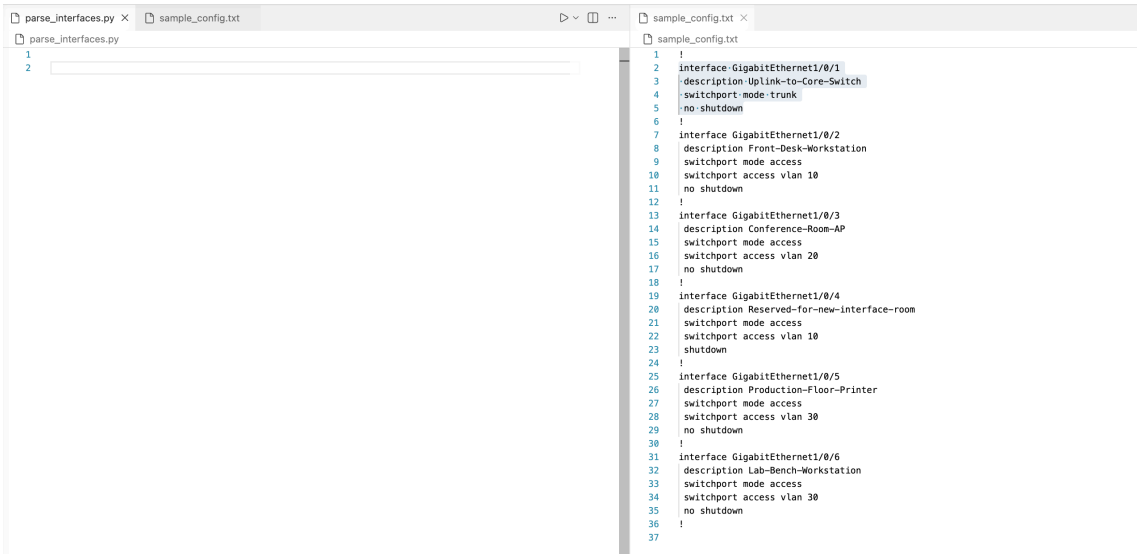
Further reading:
- CiscoConfparse documentation - https://www.pennington.net/py/ciscoconfparse/
- Cisco IOS configuration fundamentals - https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/fundamentals/configuration/15mt/fundamentals-15-mt-book.html

---

Hey, I'm Ivan. I work in a Network Operations Center, and a lot of my job is making changes to switch ports; VLAN moves, port security tweaks, that kind of thing. Requests come in batches, and the usual workflow is: copy the running-config into a spreadsheet, eyeball it, type commands. Slow and error-prone. So for my term project I built a Python tool that turns a Cisco config into structured data. Today I'll show you the core of it: the part where unstructured text becomes a list of Python dictionaries.

Starting point: an empty parse_interfaces.py next to a small sample config with six interfaces. CiscoConfParse is already installed via pip install ciscoconfparse.

Goal: a function parse_interfaces(config_path) that returns a list of dictionaries, one per interface, with name, description, mode, vlan, and admin_state.

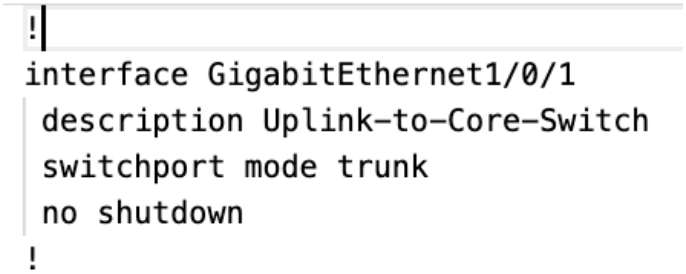


Starting point - empty parse_interfaces.py next to the sample config.

Before any code, look at the shape of one interface block:

```
interface GigabitEthernet1/0/3
 description Conference-Room-AP
 switchport mode access
 switchport access vlan 20
 no shutdown
!
```

The interface line is a *parent.* Underneath, indented by one space, are *child* lines. The exclamation mark closes the block. If we tried to parse this by hand we'd be tracking indentation, blank lines, when one block ends. It's doable but painful. Guidance from our Professor Knowles was to stay close to raw text before reaching for libraries, so notice what I did: I read the structure first, named what I needed, then picked the tool. Try to think of the library not as a magic wand but as a way to skip the indentation bookkeeping we already understand.



parse_interfaces.py > ...

```python
from ciscoconfparse import CiscoConfParse
from loguru import logger
logger.remove()  # silence CiscoConfParse's startup info log

def parse_interfaces(config_path):
    parse = CiscoConfParse(config_path)
    results = []

    for intf in parse.find_objects(r"^interface "):
        info = {
            "name": intf.text.replace("interface ", ""),
            "description": "",
            "mode": "",
            "vlan": "",
            "admin_state": "up",
        }

        for child in intf.children:
            line = child.text.strip()

            if line.startswith("description "):
                info["description"] = line.replace("description ", "")
            elif line == "switchport mode access":
                info["mode"] = "access"
            elif line == "switchport mode trunk":
                info["mode"] = "trunk"
            elif line.startswith("switchport access vlan "):
                info["vlan"] = line.replace("switchport access vlan ", "")
            elif line == "shutdown":
                info["admin_state"] = "down"

        results.append(info)

    return results

```

The finished parse_interfaces function in VS Code.

Walking through it, the import gives us CiscoConfParse, which reads the file and builds a tree that knows which lines are children of which parents. The two loguru lines just suppress an INFO log that CiscoConfParse prints on startup; not strictly necessary, but it keeps the output clean. find_objects(r"^interface ") returns every parent line that starts with ‘interface ’ followed by a space; the caret(^) means “start of line,” so we don’t accidentally match the word “interface” inside a description. Look at interface 1/0/4 in the sample, its description has “interface” in it, but the anchor keeps it from getting picked up as a parent.

```
interface GigabitEthernet1/0/4
 description Reserved-for-new-interface-room
 switchport mode access
 switchport access vlan 10
 shutdown
```

For each parent, I build a dictionary with sensible defaults. admin_state defaults to “up.” In most Cisco switches, a port is up unless explicitly shut down, so we only flip it if we see a shutdown line.

The inner loop is the parent-child walk. intf.children gives every child line indented under that interface. I strip whitespace, then match. Two patterns: startswith when there’s a value to grab like description text or a VLAN number and == when the whole line is the signal, like switchport mode access.

Run it with pprint to see the output:

```
if __name__ == "__main__":
    from pprint import pprint
    pprint(parse_interfaces("sample_config.txt"))
```

```
[{'admin_state': 'up',
  'description': 'Uplink-to-Core-Switch',
  'mode': 'trunk',
  'name': 'GigabitEthernet1/0/1',
  'vlan': ''},
 {'admin_state': 'up',
  'description': 'Front-Desk-Workstation',
  'mode': 'access',
  'name': 'GigabitEthernet1/0/2',
  'vlan': '10'},
 {'admin_state': 'up',
  'description': 'Conference-Room-AP',
  'mode': 'access',
  'name': 'GigabitEthernet1/0/3',
  'vlan': '20'},
 {'admin_state': 'down',
  'description': 'Reserved-for-new-interface-room',
  'mode': 'access',
  'name': 'GigabitEthernet1/0/4',
  'vlan': '10'},
 {'admin_state': 'up',
  'description': 'Production-Floor-Printer',
  'mode': 'access',
  'name': 'GigabitEthernet1/0/5',
  'vlan': '30'},
 {'admin_state': 'up',
  'description': 'Lab-Bench-Workstation',
  'mode': 'access',
  'name': 'GigabitEthernet1/0/6',
  'vlan': '30'}]
ivanmercado@Ivans-MacBook-Pro Week 15 %
```

Terminal showing the pprint output; a list of dictionaries, one per interface, with all five fields populated.

Now I can sort, filter, generate change commands, diff against a target state; all the things you can do with structured data and basically can't with a wall of text. On a real ticket, say, moving thirty ports from VLAN 10 to VLAN 50; this lets me confirm they're access ports and not trunks before generating commands. **Changing a trunk port's VLAN is how you take a floor offline.** Yes, in full bold to serve as a warning. Spreadsheets are still great for plenty of things; filtering tickets, tracking changes, sharing with the team. But getting Cisco config text into a clean spreadsheet structure is the messy part, and that's the step this tool replaces.

To recap: we used CiscoConfParse to walk parent-child blocks, used startswith and equality matching to pull out fields, and built a list of dictionaries one interface at a time. Everything else my project does, generating commands, building rollbacks, flagging port-security conflicts, sits on top of this one function.

Important question: If we can use the matching logic in pure Python, then what's the use of CiscoConfParse?

Answer: CiscoConfParse handles the document structure; finding interface blocks and their children based on indentation. Python's built-in string methods handle the content matching once I have the lines. I could have parsed the indentation by hand, but it would have added ~30 lines of state-tracking code that has nothing to do with the actual extraction logic. The library lets me write code that reads like the question I'm asking: "for each interface, look at each child line, match it." That's also why the 'close to raw text' guidance still holds; child.text gives me the raw line, and I do all the matching with startswith and ==. The library isn't doing the parsing decisions; it's doing the navigation.

If you want to go further, read the CiscoConfParse docs for richer matching, try adding a port-security field yourself using the same parent-child pattern, and if you build something like this for real work, write tests against a config file you control. I learned that one the hard way.

Thanks for reading!

parse_interfaces.py | sample_config.txt

```python
from ciscoconfparse import CiscoConfParse
from loguru import logger
logger.remove()  # silence CiscoConfParse's startup info log

def parse_interfaces(config_path):
    parse = CiscoConfParse(config_path)
    results = []

    for intf in parse.find_objects(r"^interface "):
        info = {
            "name": intf.text.replace("interface ", ""),
            "description": "",
            "mode": "",
            "vlan": "",
            "admin_state": "up",
        }

        for child in intf.children:
            line = child.text.strip()

            if line.startswith("description "):
                info["description"] = line.replace("description ", "")
            elif line == "switchport mode access":
                info["mode"] = "access"
            elif line == "switchport mode trunk":
                info["mode"] = "trunk"
            elif line.startswith("switchport access vlan "):
                info["vlan"] = line.replace("switchport access vlan ", "")
            elif line == "shutdown":
                info["admin_state"] = "down"

        results.append(info)

    return results

if __name__ == "__main__":
    from pprint import pprint
    pprint(parse_interfaces("sample_config.txt"))
```

sample_config.txt

```
!
interface GigabitEthernet1/0/1
 description Uplink-to-Core-Switch
 switchport mode trunk
 no shutdown
!
interface GigabitEthernet1/0/2
 description Front-Desk-Workstation
 switchport mode access
 switchport access vlan 10
 no shutdown
!
interface GigabitEthernet1/0/3
 description Conference-Room-AP
 switchport mode access
 switchport access vlan 20
 no shutdown
!
interface GigabitEthernet1/0/4
 description Reserved-for-new-interface-room
 switchport mode access
 switchport access vlan 10
 shutdown
!
interface GigabitEthernet1/0/5
 description Production-Floor-Printer
 switchport mode access
 switchport access vlan 30
 no shutdown
!
interface GigabitEthernet1/0/6
 description Lab-Bench-Workstation
 switchport mode access
 switchport access vlan 30
 no shutdown
!
```

Finishing point - our parser next to the sample configs.